Лабораторна робота №4

Тема: Інтеграція із зовнішніми Web API та автоматизоване тестування Python-коду

Мета: Навчитися отримувати відомості про вразливості через Web API, перевіряти відповіді та обробляти мережеві помилки; відокремлювати отримання даних від їх аналізу; писати відтворювані тести за допомогою pytest для звичайних, граничних і помилкових випадків.

У цій роботі ви створите консольну утиліту для перегляду запису про вразливість у **Національній базі даних вразливостей** (**National Vulnerability Database, NVD**). Утиліта шукатиме запис за ідентифікатором **CVE** (**Common Vulnerabilities and Exposures**), застосовуватиме правило вашого варіанта та формуватиме JSON-звіт. Аргументи прийматиме **інтерфейс командного рядка** (**command-line interface, CLI**). Тести перевірятимуть цю саму програму без звернень до NVD.

Перед виконанням опрацюйте лекції 10 і 14: HTTP, Requests, обробку винятків, pytest і підміну залежностей. Також знадобляться JSON, функції, модулі, `argparse` і `pathlib` із попередніх тем. Основні прийоми тестування коротко пояснено нижче.

План виконання роботи

1. Підготуйте директорію роботи в наявному репозиторії.
2. Реалізуйте клієнт NVD API та читання синтетичної відповіді з файла.
3. Перевірте структуру відповіді, виберіть оцінку CVSS і реалізуйте правило свого варіанта.
4. Створіть автоматизовані тести обробки даних, HTTP-відповідей і командного інтерфейсу.
5. Запустіть тести, перевірте програму локально та підготуйте звіт.

---

1. Підготовка проєкту

1.1. Репозиторій і залежності

Продовжуйте роботу в репозиторії `cybersecurity-python-labs`. Збережіть попередні роботи й використовуйте той самий номер варіанта з `shared/student.py`: у цій роботі передбачено **15 варіантів**. Код ЛР №4 має працювати незалежно від застосунку та бази даних ЛР №3.

Усі команди нижче виконуйте **з кореня репозиторію**, в активованому віртуальному середовищі з **Python 3.12 або новішим**:

```
pip install requests pytest ruff
```

Приклади розраховані на Requests 2.x. Додайте нові залежності до спільного `requirements.txt`, зберігши попередні пакети; зафіксуйте версії, з якими перевірили роботу. `json`, `math`, `os`, `re`, `argparse`, `pathlib` та `unittest.mock` входять до стандартної бібліотеки Python.

1.2. Структура роботи

```
cybersecurity-python-labs/
├── requirements.txt
├── shared/
│   ├── __init__.py
│   └── student.py
└── labs/
    ├── __init__.py
    ├── lab01/
    ├── lab02/
    ├── lab03/
    └── lab04/
        ├── __init__.py
        ├── main.py             # Аргументи, виклик функцій, повідомлення
        ├── api_client.py       # Заголовки та один GET-запит
        ├── analysis.py         # Перевірка даних і правило варіанта
        ├── data/
        │   └── nvd_sample.json # Синтетична відповідь для запуску без мережі
        ├── tests/              # Тести аналізу, клієнта та CLI
        └── README.md           # Встановлення, запуск, правило і тести
```

Створіть порожні `__init__.py`, якщо їх немає. Використовуйте пакетні імпорти, наприклад `from labs.lab04 import api_client`, і запуск через `python -m`; не змінюйте `sys.path` вручну. Імпорт модулів не повинен запускати програму, читати файли чи виконувати HTTP-запити.

Збережіть синтетичну відповідь NVD як `labs/lab04/data/nvd_sample.json`. Це штучний запис `CVE-2099-10001`, призначений лише для локальної перевірки. Його не потрібно шукати в NVD.

Доповніть `.gitignore` правилами `.pytest_cache/`, `__pycache__/` та `.venv/`, якщо їх ще немає. Синтетичні дані й тести додайте до Git; справжні ключі доступу не зберігайте у файлах проєкту.

---

2. Необхідні теоретичні відомості

2.1. Запит до NVD API

Прикладний програмний інтерфейс (**application programming interface, API**) визначає адресу, параметри та формат обміну із сервісом. Для цієї роботи використовуйте NVD CVE API 2.0 з фіксованою адресою:

```
https://services.nvd.nist.gov/rest/json/cves/2.0
```

Клієнт виконує `GET` і передає `cveId` через аргумент Requests `params`, наприклад `params={"cveId": "CVE-2021-44228"}`. Цей пошук повертає обгортку з масивом `vulnerabilities`, а не безпосередньо один запис. Для точного ідентифікатора очікуємо нуль або один елемент. Порожній результат пошуку та HTTP-помилка — різні ситуації.

Для читання публічних даних передбачте запуск без ключа. Якщо змінна середовища `NVD_API_KEY` містить непорожнє значення, передайте його в HTTP-заголовку `apiKey`. Ключ не є параметром URL і не використовує схему Bearer. Ключ у URL може потрапити до журналів або історії браузера навіть під час використання HTTPS; заголовки із секретами також не можна записувати в журнали. Справжній ключ у цій роботі **необов'язковий**: його читання та передавання потрібно перевірити локально з вигаданим значенням.

Виконуйте лише поодинокі ручні запити. Перед ними перегляньте правила використання NVD API; не запускайте пакетне завантаження і циклічне опитування. Для розроблення та захисту використовуйте локальні дані. У разі недоступності сервісу чи обмеження запитів зафіксуйте причину й продемонструйте локальний режим.

Таймаут (**timeout**) задає межу очікування під час мережевої операції. У цій роботі передавайте `timeout=(3.05, 10)`: час встановлення з'єднання та очікування читання. Це не загальний ліміт тривалості запиту: таймаут читання обмежує очікування між отриманням даних, а не час завантаження всієї відповіді. Перенаправлення вимкніть через `allow_redirects=False`, перевірку TLS-сертифіката залиште ввімкненою. Довільну адресу API з аргументів командного рядка не приймайте.

Джерела контракту: опис NVD CVE API, офіційна JSON-схема відповіді та JSON-схема CVSS v3.1. Перевірка використаних полів у цій роботі не є реалізацією повного валідатора цих схем.

2.2. Що означає оцінка CVSS

Загальна система оцінювання вразливостей (**Common Vulnerability Scoring System, CVSS**) описує характеристики вразливості та її технічну серйозність. Базова оцінка лежить у межах від 0 до 10. Вона не визначає сама по собі ризик для конкретної організації і не підтверджує, що її система містить вразливий компонент.

У межах роботи аналізуйте **лише CVSS v3.1** з `metrics.cvssMetricV31`. NVD може містити інші версії CVSS; їхня наявність не означає наявності оцінки v3.1. Обчислювати оцінку з вектора або переносити оцінку іншої версії не потрібно.

Оцінок v3.1 може бути декілька: від різних джерел. Зберігайте у звіті джерело вибраної оцінки, якщо воно є. Правило вибору наведено в завданні 2; це домовленість нашої програми, а не універсальна вимога NVD.

Для варіантів використовуються такі поля `cvssData`:

Поле	Зміст	Допустимі значення
`attackVector`	Спосіб доступу для атаки	`NETWORK`, `ADJACENT_NETWORK`, `LOCAL`, `PHYSICAL`
`attackComplexity`	Складність умов атаки	`LOW`, `HIGH`
`privilegesRequired`	Привілеї, потрібні до атаки	`NONE`, `LOW`, `HIGH`
`userInteraction`	Потреба у взаємодії користувача	`NONE`, `REQUIRED`
`scope`	Чи виходить вплив за межі повноважень уразливого компонента	`UNCHANGED`, `CHANGED`
`confidentialityImpact`	Вплив на конфіденційність	`NONE`, `LOW`, `HIGH`
`integrityImpact`	Вплив на цілісність	`NONE`, `LOW`, `HIGH`
`availabilityImpact`	Вплив на доступність	`NONE`, `LOW`, `HIGH`

Наприклад, `privilegesRequired="NONE"` описує відсутність потрібних попередніх привілеїв. Це не те саме, що `userInteraction="NONE"`. Значення і межі застосування метрик пояснено у специфікації FIRST CVSS v3.1.

2.3. Від відповіді до висновку

Успішний HTTP-статус ще не підтверджує придатності даних. Послідовно перевіряйте:

1. Отримано відповідь без мережевого винятку.
2. Статус відповідає очікуваному `200`.
3. Тіло успішно декодується як JSON.
4. Потрібні поля мають очікувану структуру, типи та допустимі значення.
5. Даних достатньо для застосування правила.

Перевірка вхідних даних (**validation**) потрібна також на межі із зовнішнім сервісом. Наприклад, рядок `"9.8"` не є JSON-числом, а `true` не є оцінкою CVSS. У Python `bool` є підкласом `int`, тому лише `isinstance(score, (int, float))` для цієї перевірки недостатньо. Відхиляйте також нескінченність, `NaN` і значення поза діапазоном 0–10.

Відсутність необов'язкового поля та його неправильний тип обробляйте окремо. Відсутність оцінки означає брак даних. Неправильна структура наявної оцінки означає помилку даних. Не замінюйте жодну з цих ситуацій нулем.

2.4. Автоматизовані тести

Модульний тест (**unit test**) перевіряє окрему поведінку: наприклад, вибір оцінки або рішення на межі порога. **Інтеграційний тест** (**integration test**) перевіряє взаємодію частин: наприклад, читання JSON-файла, аналіз і формування звіту. Для такого локального тесту не потрібен інтернет.

pytest знаходить файли `test_*.py` і функції `test_*`. Усередині тесту `assert` порівнює отриманий результат з очікуваним. **Параметризація** (**parametrization**) дозволяє виконати одну перевірку для кількох наборів входів.

Невеликий навчальний приклад, окремий від розв'язку роботи:

```
import pytest


def reaches_limit(count, limit):
    return count >= limit


@pytest.mark.parametrize("count, expected", [(2, False), (3, True), (4, True)])
def test_reaches_limit(count, expected):
    assert reaches_limit(count, 3) is expected
```

Збережіть цей приклад у тимчасовому файлі `test_example.py` і запустіть `python -m pytest test_example.py -q`. Очікувано буде `3 passed`. Рівність порогу тут перевіряється окремо: заміна `>=` на `>` має порушити один тест. Після ознайомлення тимчасовий файл до роботи не включайте.

Фікстура (**fixture**) готує дані для тесту. Для спільної синтетичної відповіді можна використати `@pytest.fixture`, для тимчасових файлів — `tmp_path`, а для перевірки виведення — `capsys`. Фікстура `capsys` перехоплює `stdout` і `stderr` під час тесту: через `capsys.readouterr()` можна окремо перевірити результат і повідомлення про помилку, не змішуючи їх із виведенням інших тестів. Неправильний вхід перевіряйте через `with pytest.raises(ValueError):`, очікуючи конкретний тип винятку.

Тестовий двійник (**test double**) підмінює залежність контрольованою поведінкою. Фікстура `monkeypatch` може замінити `requests.get` або функцію вашого модуля, а також тимчасово задати змінну середовища. Підмінюйте ім'я там, де код його шукає: для `import requests` у `api_client.py` це `api_client.requests.get`, а для `from requests import get` — `api_client.get`.

Підміна функції отримання даних у тесті CLI перевіряє взаємодію модулів. Щоб перевірити сам HTTP-клієнт, підміняйте нижчу залежність — виклик Requests — і викликайте справжню функцію клієнта.

---

3. Завдання

Виконайте всі три завдання для **свого** варіанта. Мережевий клієнт і формат звіту спільні; індивідуальним є правило аналізу з розділу 4.

Завдання 1. Отримання відповіді та командний інтерфейс

У `api_client.py` реалізуйте `build_headers()` і `fetch_response(cve_id)`; у `main.py` — `load_response(path)` та `main(argv=None)`. `main` повертає код завершення, а виклик під `if __name__ == "__main__":` передає його в `SystemExit`.

Вимоги до інтерфейсу:

- `--cve` — обов'язковий ідентифікатор. Перевіряйте його повний збіг з `CVE-[0-9]{4}-[0-9]{4,}`. Регістр має значення; рік існування CVE за цим шаблоном не перевіряється.
- `--input` — необов'язковий шлях до локальної відповіді NVD. Якщо його задано, читайте UTF-8 JSON-файл і **не виконуйте мережевих запитів**. Ідентифікатор `--cve` все одно потрібний для перевірки відповіді.
- Номер варіанта імпортуйте з `shared.student.VARIANT_NUMBER`. На початку запуску перевірте, що це ціле число від 1 до 15, яке не є `bool`. Окремий аргумент для зміни варіанта не потрібний.
- На успішному запуску виводьте у `stdout` лише один JSON-об'єкт звіту з `ensure_ascii=False`. Пояснення помилок спрямовуйте у `stderr`, без трасування стека.
- Код завершення `0` означає, що дані опрацьовано й сформовано звіт, зокрема зі станом `not_found` або `insufficient_data`. Код `1` означає помилку отримання, читання чи перевірки даних. Код `2` використовуйте для неправильних аргументів або номера варіанта.

Для помилок аргументів допустима стандартна поведінка `argparse`: повідомлення у `stderr` і `SystemExit(2)` замість повернення з `main()`. Такий виклик у тесті перевіряйте через `pytest.raises(SystemExit)` і значення `exception.value.code`. Для перевірки іншого номера варіанта тимчасово підмініть імпортоване ім'я в модулі `main` через `monkeypatch`.

`build_headers()` повертає заголовок `Accept: application/json` та, якщо ключ задано, `apiKey`. Зчитуйте ключ через `os.getenv()` під час виклику функції, а не при імпорті. Відсутнє, порожнє або заповнене лише пробілами значення означає запуск без ключа. Непорожнє значення очистіть від крайніх пробілів.

`fetch_response()` виконує один GET-запит за правилами розділу 2.1, викликає `raise_for_status()`, перевіряє статус `200`, декодує тіло через `response.json()` і повертає отриманий об'єкт. Перенаправлення не приймайте. Семантичну перевірку даних виконує завдання 2.

Обробіть помилки з'єднання й таймауту, HTTP-помилки та неочікувані статуси, неправильний JSON, помилки читання файла й порушення структури даних. Для кожної групи виведіть зрозуміле повідомлення у `stderr`. HTTP-помилка, зокрема `404`, не означає, що пошук CVE повернув порожній результат. Не використовуйте `except Exception` для приховування помилок програми.

Не виводьте ключ, заголовки, повний текст мережевого винятку чи тіло помилкової відповіді. Для HTTP-помилки достатньо власного повідомлення та коду статусу. Автоматичні повтори, завантаження посилань із CVE й перевірки вразливих систем до роботи не входять.

Завдання 2. Перевірка даних і аналіз за варіантом

У `analysis.py` реалізуйте дві функції:

- `extract_record(payload, cve_id)` — перевіряє відповідь і повертає внутрішній запис зі знайденими даними або `None`, якщо запису немає.
- `build_report(record, cve_id, variant)` — застосовує правило та повертає словник звіту. Формат внутрішнього запису оберіть самостійно й опишіть у docstring.

Обидві функції працюють із переданими аргументами: не читають файлів, змінних середовища чи поточного часу, не виконують HTTP-запитів і нічого не друкують. Вони не повинні змінювати отриманий `payload`. За порушення наведеного нижче контракту піднімайте `ValueError`. Перевірку поля конкретного варіанта виконуйте у `build_report`, яка отримує номер варіанта.

Перевірка відповіді та вибір оцінки:

1. Відповідь має бути словником із `vulnerabilities` — списком із нуля або одного елемента. Порожній список означає, що запис не знайдено. Для одного елемента перевірте, що він містить словник `cve`, а `cve.id` збігається із запитаним ідентифікатором.
2. Якщо `vulnStatus` відсутній, збережіть `None`; якщо наявний — це непорожній рядок. Необов'язковий `descriptions` має бути списком об'єктів із рядками `lang` і `value`. Виберіть перший непорожній опис англійською або `None`, якщо його немає.
3. Для статусу `Rejected` збережіть доступні статус і опис та сформуйте `rejected` без аналізу оцінок. Якщо статус відсутній, сформуйте `insufficient_data` без аналізу оцінок. Для цих станів поля оцінки та `observed` мають значення `null`.
4. За наявного іншого статусу відсутні `metrics`, `cvssMetricV31` або порожній список означають брак оцінки. Якщо `metrics` чи `cvssMetricV31` присутні, вони мають бути відповідно словником і списком; `null` або інший тип є помилкою. Елементи непорожнього списку мають бути словниками. Виберіть перший елемент із `type="Primary"`, а якщо його немає — перший елемент списку. Інші оцінки не аналізуйте.
5. У вибраному елементі `cvssData` має бути словником. `baseScore` має бути скінченним числом від 0 до 10, не `bool`. Поле свого варіанта перевірте за допустимими значеннями з таблиці розділу 2.2: відсутнє поле означає брак даних, а `null` чи недопустиме значення — помилку.
6. Для полів звіту використовуйте `cvssData.version`, `cvssData.vectorString` та `source` вибраної оцінки: якщо поле відсутнє, запишіть `None`; якщо присутнє — перевірте, що це непорожній рядок. Якщо `version` присутнє, воно має дорівнювати `"3.1"`. Інші поля ігноруйте.

Некоректне наявне поле не підмінюйте відсутнім. Не перераховуйте оцінку з вектора та не аналізуйте інші версії CVSS. Значення для правила беріть із тієї самої вибраної оцінки.

Формат звіту: завжди повертайте такі ключі; відсутні значення подавайте як JSON `null`:

Ключ	Значення
`cve_id`	Запитаний ідентифікатор
`variant`	Номер варіанта
`description`	Англійський опис або `null`
`vuln_status`	Статус NVD або `null`
`cvss_version`	`"3.1"` або `null`
`score`	Вибрана оцінка або `null`
`score_source`	Джерело вибраної оцінки або `null`
`vector`	Вектор вибраної оцінки або `null`
`rule`	Об'єкт із `threshold`, `field`, `expected` за варіантом
`observed`	Значення поля `rule.field` у вибраній оцінці або `null`
`decision`	Один зі станів із таблиці нижче

Застосовуйте стани в такому порядку:

Умова	`decision`
Запис не знайдено	`not_found`
`vulnStatus` дорівнює `Rejected`	`rejected`
Статус відсутній, оцінки v3.1 немає або потрібного поля варіанта немає	`insufficient_data`
Оцінка **не менша** за поріг **і водночас** потрібне поле дорівнює заданому значенню	`review`
Даних достатньо, але поєднання умов не виконується	`other`

Для `not_found` усі поля даних запису мають бути `null`; `cve_id`, `variant` і `rule` залишаються заповненими. Для `rejected` та відсутнього статусу збережіть статус і опис, але не аналізуйте оцінки. Наприклад, `Rejected` дає `rejected` навіть за наявності високої оцінки; відсутні оцінки за статусу `Analyzed` дають `insufficient_data`. Стани `other` і `insufficient_data` **не означають, що**

вразливість безпечна. Стан `review` означає відповідність навчальному правилу, а не готовий пріоритет усунення в реальній системі.

Завдання 3. Автоматизоване тестування

Створіть тести до свого клієнта, аналізу та CLI. Кількість файлів і функцій не встановлюється. Перевірте такі сценарії для свого варіанта:

- Оцінка нижче порога, на порозі й вище порога за відповідної ознаки; невідповідна ознака за високої оцінки. Порівнюйте `decision` з явним очікуваним значенням.
- Запис не знайдено; запис відхилено; оцінка або потрібне поле відсутні; некоректний тип чи значення потрібного поля спричиняє помилку.
- Перша оцінка `Primary` має пріоритет над попередньою `Secondary`; за відсутності `Primary` вибирається перша оцінка.
- Успішний запит і мережеву або HTTP-помилку перевіряйте підміною `requests.get`; переконайтеся, що передано `cveId` і таймаут. Перевірте заголовки з вигаданим ключем і без нього.
- Локальний запуск із JSON-файлом формує звіт і не звертається до мережі. Некоректний JSON або відсутній файл завершується повідомленням про помилку.

Для перевірки межі можна змінити лише `baseScore` у копії синтетичної відповіді: це тест порівняння, а не перерахунок CVSS. Тести мають виконуватися незалежно один від одного і без справжніх мережевих запитів.

У `tests/conftest.py` додайте автоматичну заборону справжніх запитів Requests і видалення успадкованого ключа перед кожним тестом:

```
import pytest


@pytest.fixture(autouse=True)
def isolated_environment(monkeypatch):
    monkeypatch.delenv("NVD_API_KEY", raising=False)
    monkeypatch.delattr("requests.sessions.Session.request")
```

Тест, який випадково дійде до справжньої мережі через Requests, завершиться помилкою. Для тестів клієнта підміняйте `requests.get` на двійник; фікстура не заважає такому виклику. Для тестів ключа задавайте вигадане значення вже всередині тесту. Не обходьте цю заборону іншою HTTP-бібліотекою. Приклад відповідає рекомендації pytest щодо заборони мережевих операцій Requests.

Для підміни успішної HTTP-відповіді можна використати `Mock` зі стандартної бібліотеки. Приклад перевіряє справжню `fetch_response()`, яку ви реалізуєте в завданні 1:

```
from unittest.mock import Mock

import requests

from labs.lab04 import api_client


def test_fetch_uses_expected_request(monkeypatch):
    payload = {"startIndex": 0, "totalResults": 0, "vulnerabilities": []}
    response = Mock(spec=requests.Response)
    response.status_code = 200
    response.headers = {"Content-Type": "application/json; charset=utf-8"}
    response.json.return_value = payload
    fake_get = Mock(return_value=response)
    monkeypatch.setattr(api_client.requests, "get", fake_get)

    assert api_client.fetch_response("CVE-2099-10001") == payload
    fake_get.assert_called_once_with(
        "https://services.nvd.nist.gov/rest/json/cves/2.0",
        params={"cveId": "CVE-2099-10001"},
        headers={"Accept": "application/json"},
        timeout=(3.05, 10),
        allow_redirects=False,
    )
    response.raise_for_status.assert_called_once_with()
```

У цьому тесті `response.json()` повертає вже підготовлені дані. Для тесту неправильного JSON задайте `side_effect=requests.exceptions.JSONDecodeError("Invalid JSON", "{", 1)`. Для HTTP-помилки підмініть поведінку `raise_for_status()`, передавши в `HTTPError` відповідь через аргумент `response`. Саме встановлення `status_code=500` у `Mock` не змусить його автоматично підняти виняток: навіть із `spec` метод `raise_for_status()` є двійником і не виконує логіку справжньої відповіді. Для імітації помилки задайте йому `side_effect` з потрібним винятком.

Очікувані результати тестів задавайте явно. Не обчислюйте їх викликом тієї самої функції або копією її умов: такий тест може повторити помилку реалізації. У фікстурах створюйте новий словник або його глибоку копію для кожного тесту.

Додатково: тимчасово замініть у правилі `>=` на `>` і перевірте, чи тест рівності порогу виявляє помилку. Після цього відновіть правильну умову та повторіть тести. Цей експеримент необов'язковий; у коді для здачі навмисної помилки бути не повинно.

---

4. Індивідуальні варіанти

Умова кожного варіанта має однакову форму: `score >= T` **і** `cvssData[field] == expected` у **тій самій вибраній оцінці**. Не поєднуйте число від одного джерела з ознакою від іншого. Пороги є навчальними параметрами, а не новими категоріями CVSS.

Варіант	Напрям відбору для ручного огляду	Поріг `T`	Поле `field`	Значення `expected`
1	Мережевий вектор атаки	7.0	`attackVector`	`NETWORK`
2	Низька складність умов атаки	8.0	`attackComplexity`	`LOW`
3	Відсутність потрібних попередніх привілеїв	7.0	`privilegesRequired`	`NONE`
4	Відсутність потрібної взаємодії користувача	8.0	`userInteraction`	`NONE`
5	Високий вплив на конфіденційність	7.0	`confidentialityImpact`	`HIGH`
6	Високий вплив на цілісність	7.0	`integrityImpact`	`HIGH`
7	Високий вплив на доступність	7.0	`availabilityImpact`	`HIGH`
8	Зміна меж впливу	9.0	`scope`	`CHANGED`
9	Локальний вектор атаки	5.0	`attackVector`	`LOCAL`
10	Низькі потрібні попередні привілеї	6.0	`privilegesRequired`	`LOW`
11	Потрібна взаємодія користувача	6.0	`userInteraction`	`REQUIRED`
12	Низький вплив на конфіденційність	6.0	`confidentialityImpact`	`LOW`
13	Низький вплив на цілісність	6.0	`integrityImpact`	`LOW`
14	Низький вплив на доступність	6.0	`availabilityImpact`	`LOW`
15	Висока складність умов атаки	8.0	`attackComplexity`	`HIGH`

Синтетичний файл спільний для всіх варіантів. За ним очікується `review` для варіантів **1–7** та `other` для варіантів **8–15**. Для повної перевірки свого правила створіть у тестах додаткові відповіді: в одному демонстраційному файлі неможливо показати всі переходи й помилки.

Випадки, у яких для перевірки межі ви змінюєте лише `baseScore`, є ізольованими тестами логіки порівняння. Вони не представляють заново розраховану оцінку CVSS для наведеного вектора.

---

5. Запуск і перевірка результатів

5.1. Приклад виконання скрипта

Спочатку перевірте локальний режим:

```
python -m labs.lab04.main --cve CVE-2099-10001 --input labs/lab04/data/nvd_sample.json
```

Для варіанта 1 очікується такий JSON-об'єкт; порядок ключів і відступи не є вимогою:

```
{
  "cve_id": "CVE-2099-10001",
  "variant": 1,
  "description": "Synthetic vulnerability for offline API testing.",
  "vuln_status": "Analyzed",
  "cvss_version": "3.1",
  "score": 9.8,
  "score_source": "nvd@nist.gov",
  "vector": "CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H",
  "rule": {
    "threshold": 7.0,
    "field": "attackVector",
    "expected": "NETWORK"
  },
  "observed": "NETWORK",
  "decision": "review"
}
```

Після локальних перевірок виконайте один ручний запит до NVD, якщо сервіс доступний:

```
python -m labs.lab04.main --cve CVE-2021-44228
```

Дані NVD змінюються, тому очікувану оцінку чи стан для живого запиту не закріплюйте в автоматичних тестах. Запишіть дату, команду та фактично отриманий результат. За недоступності сервісу зазначте це у звіті; локальні перевірки залишаються обов'язковими.

5.2. Тести та стиль коду

Запустіть тести роботи:

```
python -m pytest labs/lab04/tests -q
```

Для відтворення окремої помилки запустіть pytest з ідентифікатором тесту з його виводу та опцією `-v`.

Використовуйте наявні налаштування Ruff; не вимикайте правила для приховування помилок нової роботи:

```
ruff check labs/lab04
ruff format --check labs/lab04
```

Додавайте анотації типів і короткі docstring до основних функцій. Ruff доповнює тести: успішна перевірка стилю не підтверджує правильності правила чи обробки HTTP-помилок.

Робота готова до здачі, якщо локальний режим відтворюється за `README.md`, усі обов'язкові сценарії перевірено, звичайний запуск pytest не звертається до мережі, а всі тести проходять.

---

6. Звіт і здача роботи

У звіті наведіть:

1. Номер варіанта, правило відбору та коротке пояснення використаної метрики CVSS.
2. Посилання на свій репозиторій, команду локального запуску та отриманий JSON-звіт.
3. Результати pytest і Ruff та коротке пояснення тесту на межі порога.
4. Дату, команду й результат одного запиту до NVD або причину, з якої його не вдалося виконати. Не включайте ключі чи заголовки з ними.
5. Висновки: що перевіряють локальні тести, що потребує перевірки з реальним сервісом і чому брак даних не означає відсутності небезпеки.

У `labs/lab04/README.md` опишіть встановлення залежностей, команди запуску в обох режимах, номер варіанта, формат звіту та команди перевірки. Додайте код, синтетичні дані, тести й інструкцію до Git та завантажте на GitHub. Перед комітом перевірте `git diff --cached`, щоб не включити секрети або тимчасові файли. Захист передбачає пояснення власного правила та демонстрацію одного успішного й одного помилкового сценарію тестування.

Контрольні питання

1. Чим HTTP-помилка відрізняється від успішної відповіді з порожнім результатом пошуку CVE?
2. Чому `response.json()` (декодування) і HTTP `200` не замінюють перевірку структури даних?
3. Що обмежують два значення `timeout` у Requests? Чому це не загальний час виконання запиту?
4. Як клієнт передає параметр `cveId` і ключ `apiKey`? Чому ключ не потрібно включати до URL?
5. Де потрібно підмінювати функцію через `monkeypatch`, щоб перевірявся справжній код клієнта?
6. Чому тест із підміною функції всього клієнта не перевіряє його таймаут і заголовки?
7. Як `tmp_path`, `capsys` і автоматична фікстура (isolated_environment) допомагають ізолювати тести?
8. Чому недостатньо змінити `status_code` у `Mock`, щоб отримати HTTP-виняток?

Рекомендовані джерела

- NVD CVE API 2.0.
- Початок роботи з NVD API та обмеження використання.
- Офіційна JSON-схема NVD CVE API.
- Офіційна JSON-схема CVSS v3.1.
- FIRST: специфікація CVSS v3.1.
- NVD: запитання про CVE та тлумачення CVSS.
- Requests: запити, відповіді та винятки.
- Requests: таймаути.
- pytest: початок роботи.
- pytest: параметризація.
- pytest: підміна залежностей і змінних середовища.
- pytest: тимчасові файли.
- pytest: перевірка stdout і stderr.
- Python: unittest.mock — документація з частковим українським перекладом.
- Python: json — документація з частковим українським перекладом.
- Python: os.getenv — документація з частковим українським перекладом.
- Ruff: команди перевірки.